

Monolito modular vs. microservicios: criterios de decisión para sistemas medianos desarrollados por equipos pequeños

Jose Alexis Solís Carvajal, Luis Antonio Montes de Oca Ruiz

Escuela de Informática, Universidad Nacional de Costa Rica

jose.solis.carvajal@est.una.ac.cr · luis.montesdeoca.ruiz@est.una.ac.cr

EIF509 Desarrollo de Aplicaciones Web · II Ciclo 2026 · Prof. Elberth Adrián Garro Sánchez

Resumen

Resumen—La adopción de microservicios se ha convertido en una opción por defecto en muchos proyectos web empresariales, aun cuando el sistema y el equipo que lo construye no tienen la escala que la justifica. Este artículo analiza críticamente cuándo un monolito modular es preferible a una arquitectura de microservicios en sistemas medianos desarrollados por equipos pequeños. Ambas alternativas se contrastan con cinco criterios técnicos explícitos —rendimiento, complejidad, costo, mantenibilidad y seguridad— a partir de estudios empíricos, revisiones sistemáticas, libros de referencia y casos industriales documentados. El análisis muestra que, en una sola máquina y sin miles de usuarios concurrentes, el monolito rinde mejor y escala verticalmente a menor costo; que los microservicios trasladan complejidad del código a la operación y amplían la superficie de ataque; y que su beneficio principal, el despliegue independiente, es de naturaleza organizativa y solo compensa cuando existen varios equipos autónomos. El análisis se aplica a Gastonomo, un sistema web de control de gastos personales construido por un equipo de dos personas, cuya conciliación de comprobantes escribe en cinco tablas dentro de una única transacción. Se decide no adoptar microservicios y evolucionar la arquitectura actual en capas hacia un monolito modular organizado por contextos de negocio, con la ingesta de correos como único candidato a separarse cuando su carga lo justifique. Se concluye que el monolito modular es el punto de partida más razonable para sistemas de esta escala.

Palabras clave—monolito modular, microservicios, arquitectura de software, diseño dirigido por el dominio, aplicaciones web empresariales.

I. INTRODUCCIÓN

En la última década, los microservicios pasaron de ser una técnica emergente a convertirse, para muchos equipos, en sinónimo de arquitectura moderna. ThoughtWorks bautizó este fenómeno como *microservice envy*: la tentación de complicar una arquitectura con muchos servicios simplemente porque es una elección de moda, olvidando que los microservicios cambian complejidad de desarrollo por complejidad operativa [1]. Fowler observó un patrón persistente: casi todas las historias exitosas de microservicios comenzaron con un monolito que creció demasiado y se dividió, mientras que casi todos los sistemas construidos como microservicios desde cero terminaron en serios problemas [2]. Newman, en la segunda edición de su obra de referencia, insiste en que los microservicios no deberían ser la opción por defecto y en que un monolito bien modularizado es una arquitectura legítima, no una deuda que haya que saldar [3].

La discusión dejó de ser teórica. Su, Li y Taibi documentan, en una revisión multivocal de 2024, casos de equipos que revirtieron sus microservicios hacia un monolito —entre ellos Amazon Prime Video, Segment e InVision— y encuentran que el costo, la complejidad, la escalabilidad, el rendimiento y la organización fueron los motivos principales [4]. En paralelo, experimentos controlados muestran que, para sistemas que no tienen miles de usuarios concurrentes y que pueden escalarse verticalmente, los beneficios de migrar a microservicios no han sido suficientemente demostrados y la evidencia existente es inconsistente [5].

El problema que aborda este artículo es, por tanto, de decisión: ¿con qué criterios debe un equipo elegir entre un monolito modular y una arquitectura de microservicios cuando construye un sistema web mediano? Para efectos de este análisis, se entiende por *sistema mediano* una aplicación de negocio con decenas de entidades y algunos procesos transaccionales, que atiende de cientos a pocos miles de usuarios concurrentes y que es desarrollada por un equipo pequeño, es decir, por una o pocas células de trabajo de entre dos y algunas decenas de personas. El objetivo es contrastar ambas alternativas con criterios técnicos explícitos —rendimiento, complejidad, costo, mantenibilidad y seguridad—, respaldados por evidencia formal, y aplicar la conclusión al sistema que el equipo desarrolla en los laboratorios del curso.

El resto del artículo se organiza así: la Sección II presenta el marco conceptual y el estado del arte; la Sección III desarrolla el análisis comparativo criterio por criterio y lo sintetiza en la Tabla I; la Sección IV aplica el análisis a Gastonomo, el sistema de control de gastos del equipo, y argumenta una decisión; y la Sección V presenta los hallazgos, los límites del estudio y el trabajo futuro.

II. MARCO CONCEPTUAL

A. Microservicios

Lewis y Fowler definen el estilo de microservicios como la construcción de una aplicación en forma de un conjunto de servicios pequeños, cada uno ejecutado en su propio proceso, comunicado mediante mecanismos livianos —por lo general HTTP—, organizado alrededor de capacidades de negocio y desplegable de forma independiente, con gestión descentralizada de los datos [6]. Newman resalta que la propiedad central del estilo es el despliegue independiente: un servicio debe poder modificarse y ponerse en producción sin

coordinar el despliegue de los demás [3]. Esa independencia tiene un precio fijo que Fowler denominó *microservice premium*: el costo de administrar un conjunto de servicios distribuidos (fallas de red, versionado de contratos, monitoreo, consistencia) que reduce la productividad del equipo y que solo se recupera cuando la complejidad del sistema supera cierto umbral [7].

B. Monolito y monolito modular

Un monolito es un sistema cuya funcionalidad se despliega como una sola unidad. Newman distingue tres variantes: el monolito de proceso único, el monolito modular y el monolito distribuido; este último —varios servicios que de todos modos deben desplegarse juntos— reúne las desventajas de ambos mundos y suele ser el resultado de una descomposición prematura [3]. Su y Li definen el monolito modular como un patrón que combina la simplicidad de la estructura monolítica con ventajas propias de los microservicios: el sistema se organiza en módulos débilmente acoplados, con fronteras bien definidas y dependencias explícitas, y puede permanecer así o migrarse más adelante a servicios con menor esfuerzo que si se partiera de un monolito sin estructura [8]. En la práctica, cada módulo encapsula su modelo y sus datos y expone una interfaz pública; los demás módulos no acceden a su estado interno.

C. Contexto delimitado

El diseño dirigido por el dominio (DDD, por sus siglas en inglés) introduce el contexto delimitado (*bounded context*): la frontera dentro de la cual un modelo de dominio y su lenguaje tienen un significado único y consistente [9]. Khononov argumenta que los contextos delimitados señalan las fronteras más amplias que puede tener un servicio sin fragmentar el modelo, de modo que sirven tanto para definir módulos dentro de un monolito como para trazar, si llega el momento, los límites de los servicios [10]. Esta equivalencia es la que permite tratar el monolito modular como una etapa evolutiva y no como un callejón sin salida.

D. Consistencia de datos distribuidos

Cuando cada servicio posee su propia base de datos, las transacciones ACID (atomicidad, consistencia, aislamiento y durabilidad) dejan de abarcar el sistema completo. Richardson describe el patrón Saga como la alternativa: una secuencia de transacciones locales en la que cada paso publica un evento y, si un paso falla, se ejecutan transacciones de compensación que deshacen los anteriores; las consultas que cruzan servicios se resuelven mediante composición de interfaces de programación (API) o mediante separación de comandos y consultas (CQRS), al costo de aceptar consistencia eventual [11]. Este punto es determinante en sistemas cuyo valor depende de que los números cuadren.

E. Ley de Conway

Conway formuló en 1968 que cualquier organización que diseña un sistema produce un diseño cuya estructura es una copia de la estructura de comunicación de la organización [12]. Aplicada a esta discusión, la ley sugiere que los límites de los servicios deberían coincidir con los límites entre equipos: una

organización con tres equipos autónomos tiene argumentos para tres servicios principales, no para treinta.

F. Estado del arte reciente

La literatura reciente matiza la dicotomía. Ghemawat et al., de Google, sostienen que los microservicios confunden las fronteras lógicas (cómo se escribe el código) con las físicas (cómo se despliega) y proponen escribir la aplicación como un monolito lógico dividido en componentes, dejando que el entorno de ejecución decida cuáles se distribuyen; su prototipo redujo la latencia hasta 15 veces y el costo hasta 9 veces frente a una implementación tradicional de microservicios [13]. Su y Li identifican el monolito modular como una tendencia emergente, con marcos de trabajo y casos industriales que lo adoptan como alternativa o como paso intermedio [8]. Zhou et al., a partir de 20 entrevistas con profesionales, confirman los beneficios de los microservicios en entrega continua y escalabilidad, pero reportan dolores persistentes en la consistencia de datos, el monitoreo y la subjetividad de la descomposición en servicios [14]. Auer et al. proponen un marco de evaluación para que la decisión de migrar se base en métricas y hechos, no en intuiciones [15].

III. ANÁLISIS COMPARATIVO

El análisis contrasta un monolito modular, desplegado como una unidad pero con módulos aislados, con una arquitectura de microservicios con base de datos por servicio. Se utilizan cinco criterios técnicos y uno organizativo que la literatura señala como condicionante:

C1. Rendimiento: latencia de las operaciones, capacidad de respuesta bajo carga y eficiencia en el uso de recursos.

C2. Complejidad: cantidad de piezas que hay que desplegar, monitorear y depurar, y dificultad de mantener la consistencia de los datos.

C3. Costo: gasto de infraestructura y fracción de la capacidad de ingeniería dedicada a operar el sistema.

C4. Mantenibilidad: facilidad para modificar, probar y evolucionar el sistema sin degradar sus fronteras internas.

C5. Seguridad: superficie de ataque, control de acceso entre componentes y aislamiento ante un compromiso.

C6. Madurez organizativa: prácticas y estructura de equipos que cada alternativa presupone.

La Tabla I resume el contraste; las subsecciones siguientes presentan la evidencia que lo sustenta.

TABLA I
Comparación de alternativas según criterios técnicos

Criterio	Monolito modular	Microservicios
C1. Rendimiento	Llamadas en memoria y transacciones locales; el escalado vertical es eficiente [5].	Red y serialización en cada llamada; escala por componente, útil solo con cargas heterogéneas [13].
C2. Complejidad	Un artefacto, una base de datos y depuración local.	Muchos artefactos, Sagas y trazas distribuidas; dolores en consistencia y monitoreo [14].
C3. Costo	Infraestructura mínima; un solo proceso que operar.	Infraestructura y personal por servicio; principal causa de reversiones [4].

Criterio	Monolito modular	Microservicios
C4. Mantenibilidad	Refactorizar es barato, pero las fronteras se erosionan si no se hacen cumplir [16].	Fronteras fuertes; cambios entre servicios exigen versionar contratos; bibliotecas divergen [17].
C5. Seguridad	Un perímetro y una autenticación; un compromiso expone todo el proceso.	Más puntos de entrada y tráfico expuesto; aísla daños si los controles se operan bien [18].
C6. Madurez	Integración continua, pruebas automatizadas y disciplina modular.	Entrega continua por servicio, infraestructura como código y observabilidad [1].

A. Rendimiento (C1)

Blinowski et al. implementaron una misma aplicación web de referencia en cuatro versiones —monolito y microservicios, en Java y en C# .NET— y la sometieron a experimentos controlados en un entorno local y en dos servicios de Azure. Sus hallazgos son directos: en una sola máquina el monolito rinde mejor que su contraparte de microservicios; el escalado vertical resulta más rentable que el horizontal; y escalar horizontalmente más allá de cierto número de instancias degrada el rendimiento [5]. Los autores concluyen que el monolito es la mejor elección para sistemas que no deben atender un gran número de usuarios concurrentes. La razón es estructural: una llamada entre módulos es una invocación en memoria, mientras que entre servicios implica serialización, red y posibles reintentos; Ghemawat et al. atribuyen a esos costos buena parte de las mejoras de latencia de su prototipo [13]. Los microservicios solo ganan este criterio cuando ciertos componentes tienen perfiles de carga muy distintos —por ejemplo, uno intensivo en cómputo frente a otro intensivo en entrada/salida (E/S)— y escalearlos por separado compensa el sobrecosto de la comunicación.

B. Complejidad (C2)

ThoughtWorks advierte que los microservicios intercambian complejidad de desarrollo por complejidad operativa [1]. Esa complejidad aparece en tres frentes. Primero, en los datos: sin transacciones que crucen servicios, cada proceso de negocio que hoy termina en un único COMMIT se convierte en una Saga con compensaciones, idempotencia y estados intermedios visibles [11]. Segundo, en la operación: cada servicio agrega un artefacto que construir, desplegar, configurar y vigilar, y un fallo deja de tener una traza local para dispersarse entre procesos; los profesionales entrevistados por Zhou et al. señalan precisamente la consistencia de datos y el monitoreo como sus principales dolores [14]. Tercero, en la descomposición: decidir dónde cortar es subjetivo, y un corte equivocado entre servicios es mucho más caro de corregir que entre módulos [2], [14]. El caso de Segment ilustra el resultado acumulado: con más de 140 servicios, tres ingenieros de tiempo completo dedicaban la mayor parte de su tiempo solo a mantener el sistema en funcionamiento [17].

C. Costo (C3)

El costo es la causa más frecuente de las reversiones documentadas por Su et al.; el caso más citado es el servicio de monitoreo de audio y video de Amazon Prime Video, que redujo en 90 % sus costos de infraestructura al consolidar en un

solo proceso componentes que antes estaban distribuidos [4]. En condiciones experimentales, el prototipo de Ghemawat et al. requirió hasta nueve veces menos núcleos de procesamiento que la versión en microservicios [13], y Blinowski et al. encontraron más rentable escalar verticalmente que añadir instancias [5]. El costo no es solo de infraestructura: la prima de microservicios incluye el tiempo que el equipo deja de dedicar al dominio [7]. Tras consolidar sus servicios, Segment pasó de 32 a 46 mejoras anuales en sus bibliotecas compartidas y dejó de recibir alertas por picos de carga [17].

D. Mantenibilidad (C4)

Este es el criterio más equilibrado. Los microservicios imponen fronteras fuertes: un servicio no puede acceder al estado interno de otro porque la red se lo impide, lo que frena la erosión del diseño y permite evolucionar cada parte por separado [3]. El precio es que todo cambio que cruza una frontera exige versionar un contrato y coordinar despliegues, y que el código compartido tiende a divergir: en Segment, los servicios terminaron usando versiones distintas de las mismas bibliotecas comunes [17]. En el monolito modular la situación se invierte: refactorizar entre módulos es barato —un mismo repositorio, las mismas pruebas, un solo despliegue—, pero las fronteras se erosionan si nada las hace cumplir.

Shopify, con más de mil desarrolladores trabajando sobre uno de los mayores códigos Ruby on Rails existentes, reorganizó su monolito por dominios de negocio, aisló las dependencias detrás de interfaces públicas y construyó herramientas internas para medir y hacer cumplir el aislamiento entre componentes; su conclusión es que la solución correcta depende de la escala en la que se opera [16]. Basecamp, por su parte, sostiene un producto en seis plataformas con doce programadores sobre un único monolito [19]. La mantenibilidad, por tanto, depende menos del estilo que de que las fronteras se definan por contextos de negocio y se verifiquen automáticamente; mientras esas fronteras no sean estables, es más barato corregirlas dentro de un monolito [2].

E. Seguridad (C5)

Hannousse y Yahiouche, en un mapeo sistemático de 46 estudios primarios, concluyen que asegurar una arquitectura de microservicios es más difícil por el gran número de puntos de entrada y por el tráfico de comunicación que genera descomponer el sistema; las amenazas más tratadas son el acceso no autorizado, la exposición de datos sensibles y el compromiso de servicios individuales [18]. La guía SP 800-204 del Instituto Nacional de Estándares y Tecnología (NIST) de EE. UU. recomienda, en consecuencia, autenticar y autorizar las solicitudes en cada servicio, proteger la comunicación entre servicios, centralizar el acceso externo en una pasarela de API y aplicar mecanismos como cortacircuitos y límites de tasa [20].

Bien operados, esos controles ofrecen una ventaja real: el compromiso de un servicio queda acotado a sus datos y credenciales. Pero cada control es una pieza más que configurar y mantener, y una configuración incorrecta reabre la superficie que pretendía cerrar. En un monolito modular las llamadas internas no viajan por la red y existe un solo perímetro y un solo mecanismo de autenticación; a cambio, el compromiso del

proceso expone todos sus datos, lo que obliga a reforzar el perímetro, cifrar los datos sensibles y aplicar mínimo privilegio en la base de datos.

F. Madurez organizativa (C6)

ThoughtWorks nunca llevó los microservicios al anillo *Adopt* de su Technology Radar —los ubicó en *Assess* y luego en *Trial* entre 2012 y 2014 [21]— y mantiene *microservice envy* en *Hold*, con la advertencia de que el estilo exige una base sólida de pruebas automatizadas, entrega continua y cultura DevOps [1]. La Ley de Conway explica por qué [12]: el beneficio diferencial de los microservicios, el despliegue independiente, solo tiene valor cuando varios equipos se bloquean mutuamente al desplegar. Incluso a gran escala la granularidad excesiva termina corrigiéndose: Uber, con unos 2200 microservicios críticos, tuvo que agruparlos en dominios y capas para contener la complejidad [22].

G. Síntesis y criterios de decisión

Para un sistema mediano construido por un equipo pequeño, el monolito modular gana en rendimiento, complejidad y costo; empata en mantenibilidad siempre que sus fronteras se hagan cumplir; y presenta una superficie de ataque menor, aunque con menor aislamiento. Los microservicios ganan en despliegue independiente y en escalado selectivo, dos beneficios que solo se materializan con varios equipos o con cargas muy heterogéneas. De lo anterior se derivan cuatro preguntas de decisión que se aplican en la Sección IV:

- D1. ¿Existen varios equipos autónomos que se bloquean entre sí al desplegar?
- D2. ¿Hay componentes con perfiles de carga o requisitos de disponibilidad radicalmente distintos?
- D3. ¿Los procesos críticos del dominio pueden tolerar consistencia eventual?
- D4. ¿La organización cuenta con entrega continua, infraestructura como código y observabilidad distribuida?

Si la mayoría de las respuestas es negativa, el monolito modular es la opción por defecto, en línea con el enfoque basado en evidencia que proponen Auer et al. [15].

IV. APLICACIÓN AL DOMINIO PROPIO

A. El sistema: Gastonomo

Gastonomo es el sistema que el equipo construye en los laboratorios del curso: una aplicación web multiusuario de control de gastos personales por categorías, pensada para personas en Costa Rica. El titular de una cuenta registra compras a mano, con desglose por renglón, o vincula su buzón de Gmail u Outlook para que el sistema concilie automáticamente los comprobantes bancarios: extrae el monto, los últimos cuatro dígitos de la tarjeta, el comercio y la fecha; empareja el método de pago; resuelve el comercio y, mediante una cadena de reglas, la categoría; convierte la moneda con el tipo de cambio histórico del Banco Central de Costa Rica, con respaldo en el Ministerio de Hacienda; y actualiza el avance del presupuesto mensual. Su diferenciador es la trazabilidad: de cualquier total se puede bajar hasta el comprobante y la regla que lo originaron.

El sistema está escrito en Python 3.14 con FastAPI y SQLAlchemy 2.0. El núcleo transaccional reside en PostgreSQL, con 14 entidades cuyo esquema se versiona con migraciones Flyway, y una colección de MongoDB guarda la bitácora de trazabilidad de cada compra. Hoy es un monolito organizado en tres capas —presentación, negocio y datos, con la regla de que las dependencias solo apuntan hacia abajo— y desplegado como un único proceso del servidor Uvicorn. Su integración continua ejecuta análisis estático, pruebas unitarias con umbral de cobertura y pruebas de integración contra un PostgreSQL real. Lo desarrollan dos personas a lo largo de siete laboratorios incrementales en un ciclo lectivo.

B. Evaluación con los criterios de decisión

D1 (*equipos*). Hay un solo equipo de dos personas. Por la Ley de Conway, la estructura natural es un único artefacto desplegable; el despliegue independiente no aportaría nada, porque ambos integrantes coordinan cada entrega de todos modos.

D2 (*carga*). Los usuarios previstos son personas que consultan sus gastos y reciben decenas de comprobantes al mes; no hay miles de usuarios concurrentes. Según la evidencia de Blinowski et al., en ese régimen un solo proceso escalado verticalmente rinde mejor y cuesta menos [5]. El único componente con un perfil distinto es la ingesta de correos: periódica, intensiva en E/S y dependiente de proveedores externos.

D3 (*consistencia*). Es el criterio decisivo. La conciliación de un comprobante escribe en cinco tablas —comprobante, compra, línea de compra, regla de categorización y presupuesto— dentro de una sola transacción con un único COMMIT, y el equipo verifica con pruebas de integración que un fallo provocado a mitad del proceso no deja ninguna escritura. Si se separaran servicios de Compras, Categorización y Presupuestos con base de datos propia, esa transacción se convertiría en una Saga [11]: si fallara la actualización del presupuesto, habría que compensar eliminando la compra y desligando el comprobante, y mientras tanto el titular vería un gasto que no impacta su presupuesto, que es exactamente la inconsistencia que el dominio prohíbe.

D4 (*madurez*). El equipo cuenta con integración continua, pero no con infraestructura como código, trazas distribuidas, pasarela de API ni guardias operativas. Incorporarlas consumiría buena parte del ciclo sin aportar funcionalidad al dominio, que es justamente la prima descrita por Fowler [7].

A esto se suma la seguridad: el sistema custodia información financiera y tokens de acceso a buzones de correo. Repartir esos datos entre varios servicios multiplicaría los puntos de entrada y las credenciales entre servicios que habría que proteger [18], [20].

C. Decisión argumentada

Decisión: no adoptar microservicios y evolucionar el monolito en capas hacia un monolito modular. Las cuatro preguntas de decisión tienen respuesta negativa, y el proceso de mayor valor del dominio depende de una transacción ACID que una arquitectura distribuida obligaría a reemplazar por

compensaciones. En concreto, la arquitectura cambiaría en tres puntos:

1) *Módulos por contexto de negocio*. El código, hoy agrupado solo por capa técnica, se reorganizaría en módulos alineados con contextos delimitados —Identidad (usuarios y cuentas de correo), Catálogo (categorías, comercios, métodos de pago y reglas), Compras (registro y conciliación), Presupuestos, Ingesta (lectores de correo y analizadores de comprobantes) y Tipo de cambio—, conservando dentro de cada uno la separación en presentación, negocio y datos, tal como Shopify pasó de capas técnicas a dominios [16].

2) *Interfaces públicas verificadas*. Cada módulo expone solo sus servicios y sus objetos de comando inmutables, que el equipo ya usa como frontera del dominio, y ningún módulo podría importar los repositorios o modelos de otro. Como Python no impide esas importaciones, la regla se verificaría en la integración continua mediante contratos de importación (por ejemplo, con la herramienta `import-linter`), de modo que una violación detenga la integración en lugar de descubrirse en una revisión manual.

3) *Propiedad de los datos sin perder la transacción*. Cada módulo sería dueño de sus tablas y los demás solo accederían a ellas a través de su interfaz; pero, al compartir el proceso y la sesión de base de datos, la conciliación seguiría orquestando las cinco escrituras en una única unidad de trabajo. Esa es la ventaja que el monolito modular ofrece y los microservicios no: fronteras lógicas explícitas sin renunciar a la atomicidad [8], [13].

Costo y beneficio. El cambio es una reorganización de paquetes e importaciones, sin cambios de esquema ni de infraestructura, y puede ejecutarse módulo por módulo entre laboratorios. A cambio, el equipo gana fronteras visibles y verificadas, reduce el riesgo de que las capas se filtren —un riesgo que ya había documentado al elegir Python como lenguaje— y deja preparado el camino para extraer servicios si las condiciones cambian. El sistema, además, ya aplica consistencia eventual donde el dominio la tolera: la bitácora de MongoDB se escribe después del COMMIT y nunca bloquea la conciliación.

Candidato a extracción futura. La Ingesta es el único módulo con un perfil propio (D2). La primera medida sería ejecutarla como un proceso de trabajo separado a partir del mismo código y la misma base de datos, lo cual no la convierte en un microservicio. Solo se justificaría extraerla como servicio independiente si la sincronización de buzón degradara de forma medible la latencia de la API, o si otro equipo asumiera su desarrollo; es decir, cuando D1 o D2 dejen de ser negativas.

V. CONCLUSIONES

Hallazgos. La evidencia revisada converge: para sistemas medianos construidos por equipos pequeños, el monolito modular supera a los microservicios en rendimiento, complejidad operativa y costo, y ofrece una superficie de ataque menor. Los microservicios aportan despliegue independiente y escalado selectivo, beneficios reales pero de naturaleza organizativa, que solo se materializan con varios equipos

autónomos y con una madurez operativa que pocas organizaciones pequeñas poseen. La mantenibilidad no depende del estilo, sino de que las fronteras se tracen por contextos de negocio y se hagan cumplir automáticamente; el monolito modular permite lograrlo sin pagar la prima de la distribución. En Gastonomo, el análisis lleva a mantener un único artefacto desplegable y reorganizarlo por módulos, porque su proceso de mayor valor depende de una transacción ACID de cinco tablas.

Límites. El análisis se apoya en literatura académica y en casos publicados por las propias empresas, que pueden tener sesgos de selección; no se midió experimentalmente el comportamiento de Gastonomo bajo carga ni se comparó con una versión distribuida. El umbral de «sistema mediano» es cualitativo, y quedaron fuera la orquestación de contenedores, las plataformas sin servidor y una evaluación cuantitativa de la seguridad.

Trabajo futuro. Queda pendiente realizar pruebas de carga sobre Gastonomo para verificar si la ingesta de correos justifica un proceso separado; medir el acoplamiento entre módulos antes y después de la modularización; y evaluar, para equipos de tamaño intermedio, enfoques que separan el diseño lógico del despliegue físico, como el propuesto por Ghemawat et al. [13], o la extracción gradual de servicios a partir de un monolito modular.

REFERENCIAS

- [1] ThoughtWorks, «Microservice envy», *Technology Radar*, vol. 19, nov. 2018. [En línea]. Disponible en: <https://www.thoughtworks.com/radar/techniques/microservice-envy> [Accedido: 04-oct-2026].
- [2] M. Fowler, «MonolithFirst», martinfowler.com, 3-jun-2015. [En línea]. Disponible en: <https://martinfowler.com/bliki/MonolithFirst.html> [Accedido: 04-oct-2026].
- [3] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2021.
- [4] R. Su, X. Li y D. Taibi, «From microservice to monolith: A multivocal literature review», *Electronics*, vol. 13, n.º 8, art. 1452, 2024, doi: 10.3390/electronics13081452.
- [5] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. microservice architecture: A performance and scalability evaluation», *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [6] J. Lewis y M. Fowler, «Microservices: A definition of this new architectural term», martinfowler.com, 25-mar-2014. [En línea]. Disponible en: <https://martinfowler.com/articles/microservices.html> [Accedido: 04-oct-2026].
- [7] M. Fowler, «Microservice premium», martinfowler.com, 13-may-2015. [En línea]. Disponible en: <https://martinfowler.com/bliki/MicroservicePremium.html> [Accedido: 04-oct-2026].
- [8] R. Su y X. Li, «Modular monolith: Is this the trend in software architecture?», en *Proc. 1st Int. Workshop New Trends Softw. Archit. (SATrends), ICSE 2024*, Lisboa, Portugal, 2024, doi: 10.1145/3643657.3643911.
- [9] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, EE. UU.: Addison-Wesley, 2003.
- [10] V. Khononov, *Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy*. Sebastopol, CA, EE. UU.: O'Reilly Media, 2021.
- [11] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, EE. UU.: Manning, 2018.
- [12] M. E. Conway, «How do committees invent?», *Datamation*, vol. 14, n.º 4, pp. 28–31, abr. 1968.
- [13] S. Ghemawat, R. Grandl, S. Petrovic, M. Whittaker, P. Patel, I. Posva y A. Vahdat, «Towards modern development of cloud applications», en

- Proc. 19th Workshop Hot Topics Oper. Syst. (HotOS '23)*, Providence, RI, EE. UU., 2023, pp. 110–117, doi: 10.1145/3593856.3595909.
- [14] X. Zhou, S. Li, L. Cao, H. Zhang, Z. Jia, C. Zhong, Z. Shan y M. A. Babar, «Revisiting the practices and pains of microservice architecture in reality: An industrial inquiry», *J. Syst. Softw.*, vol. 195, art. 111521, 2023, doi: 10.1016/j.jss.2022.111521.
 - [15] F. Auer, V. Lenarduzzi, M. Felderer y D. Taibi, «From monolithic systems to microservices: An assessment framework», *Inf. Softw. Technol.*, vol. 137, art. 106600, 2021, doi: 10.1016/j.infsof.2021.106600.
 - [16] K. Westeinde, «Deconstructing the monolith: Designing software that maximizes developer productivity», *Shopify Engineering*, 21-feb-2019. [En línea]. Disponible en: <https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity> [Accedido: 04-oct-2026].
 - [17] A. Noonan, «Goodbye microservices: From 100s of problem children to 1 superstar», *Segment Engineering Blog*, 10-jul-2018. [En línea]. Disponible en: <https://www.twilio.com/en-us/blog/developers/best-practices/goodbye-microservices> [Accedido: 04-oct-2026].
 - [18] A. Hannousse y S. Yahiouche, «Securing microservices and microservice architectures: A systematic mapping study», *Comput. Sci. Rev.*, vol. 41, art. 100415, 2021, doi: 10.1016/j.cosrev.2021.100415.
 - [19] D. Heinemeier Hansson, «The majestic monolith», *Signal v. Noise*, 29-feb-2016. [En línea]. Disponible en: <https://signalvnoise.com/svn3/the-majestic-monolith/> [Accedido: 04-oct-2026].
 - [20] R. Chandramouli, «Security strategies for microservices-based application systems», National Institute of Standards and Technology, Gaithersburg, MD, EE. UU., NIST SP 800-204, ago. 2019, doi: 10.6028/NIST.SP.800-204.
 - [21] ThoughtWorks, «Microservices», *Technology Radar*, 2012–2014. [En línea]. Disponible en: <https://www.thoughtworks.com/radar/techniques/microservices> [Accedido: 04-oct-2026].
 - [22] A. Gluck, «Introducing domain-oriented microservice architecture», *Uber Engineering Blog*, 23-jul-2020. [En línea]. Disponible en: <https://www.uber.com/blog/microservice-architecture/> [Accedido: 04-oct-2026].